DESENVOLVIMENTO MOBILE · MÓDULO 10 · TRILHA KOTLIN · NÍVEL INTERMEDIÁRIO

Persistência Local

Opções de armazenamento, DataStore, Room, relações, migrações e integração com a arquitetura

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Escolher a forma de armazenamento adequada a cada tipo de dado.
- Guardar preferências do usuário com **DataStore**.
- Modelar e consultar um banco SQLite com **Room**: entidades, DAOs e banco.
- Representar relações entre tabelas e converter tipos.
- Evoluir o esquema do banco com migrações sem perder dados.
- Integrar o banco ao repositório e expor dados reativos com `Flow`.

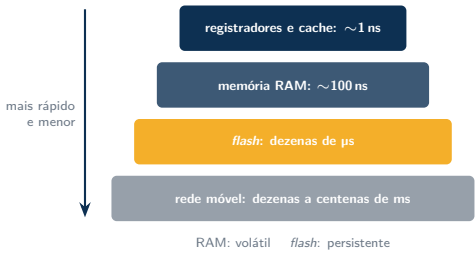
Roteiro

Neste módulo

- Fundamentos: persistência, cache e sincronização
- Opções de armazenamento
- DataStore
- Room
- Integração com a arquitetura
- Fechamento

Fundamentos: persistência, cache e sincronização

Memória e armazenamento: velocidade *versus* permanência



Persistência

Guardar dados de forma que sobrevivam ao fim do processo, ao desligar do aparelho e à falta de rede. No celular, isso significa gravar na memória *flash*.

Por que persistir em um app

O sistema encerra apps em segundo plano (Módulo 07), a rede falha e o usuário espera abrir o app e ver seus dados na hora.

A memória *flash*

Grava em blocos e se desgasta com o uso. Muitas gravações pequenas custam mais que uma grande: agrupe operações em transações (Módulo 09).

Onde o sistema guarda os arquivos do app

Finalidade	Android	iOS	Flutter (`path_provider`)
Dados internos do app	`filesDir`, `databases/`	`Library/Application Support`	`getApplicationSupportDirectory()`
Documentos do usuário	`filesDir` ou MediaStore	`Documents`	`getApplicationDocumentsDirectory()`
Cache descartável	`cacheDir`	`Library/Caches`	`getApplicationCacheDirectory()`
Temporários	`cacheDir`	`tmp`	`getTemporaryDirectory()`

Sandbox de arquivos

Cada app tem pastas privadas que outros apps não leem. Desinstalar o app apaga tudo.

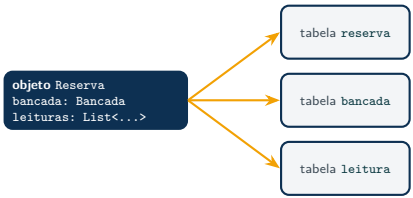
Cópia de segurança

O Android copia até 25 MB por app para a conta Google; o iOS inclui `Documents` e `Library`, exceto `Caches`.

Cache some

O sistema apaga caches quando falta espaço. Nunca guarde ali o que não puder baixar de novo.

Do objeto à tabela



referências viram chaves estrangeiras; listas viram outra tabela

Descompasso objeto-relacional

Objetos têm referências, listas e herança; tabelas têm linhas, colunas e chaves. Traduzir de um mundo para o outro é o trabalho do mapeamento.

Abordagem	Como é
SQL à mão	você escreve o SQL e converte as linhas (`sqflite`)
SQL verificado	você escreve o SQL; a ferramenta confere na compilação e gera o código (Room, Drift)
ORM completo	a biblioteca gera o SQL a partir dos objetos
Banco de objetos	guarda objetos sem tabelas (ObjectBox, Realm)

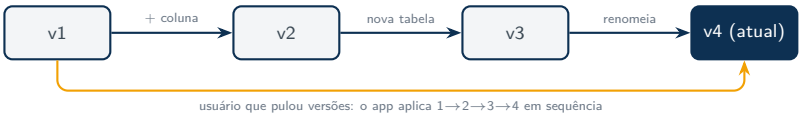
Dica

Room e Drift ficam no meio-termo: SQL explícito, mas com tipos e erros apontados antes de o app rodar.

O problema N+1

Buscar uma lista e depois uma consulta por item gera N+1 idas ao banco. Use `JOIN` ou relações carregadas de uma vez.

O esquema também tem versões



Migração

Programa que leva o banco de uma versão do esquema para a seguinte **sem perder os dados** do usuário. Cada versão publicada do app pode encontrar qualquer versão anterior do banco.

Regras de ouro

Migrações só andam para a frente e nunca são editadas depois de publicadas; teste a partir de **cada** versão antiga; apagar e recriar o banco só serve para dados de cache.

Cache: perto e rápido, mas pode estar velho

Cache

Cópia de dados guardada perto de quem usa para evitar buscá-los de novo. **Acerto**: o dado estava lá. **Falta**: é preciso ir à fonte.

Estratégia	Como funciona
Cache primeiro	usa o cache; só vai à rede se não houver
Rede primeiro	tenta a rede; usa o cache se falhar
Mostra e revalida	mostra o cache já e atualiza em seguida (*stale-while-revalidate*)
Só rede	dados que não podem estar velhos (saldo, reserva)

Invalidação

Decidir quando o cache deixou de valer: por tempo (TTL), por evento (o servidor avisou) ou por versão (*ETag*, Módulo 11).

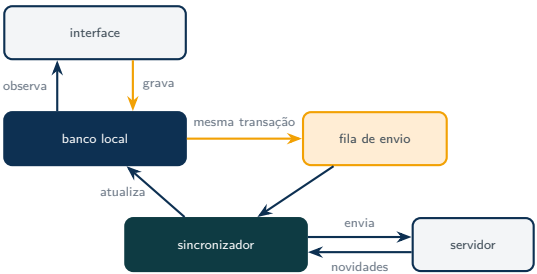
Quem sai quando enche

A política mais comum é a LRU (*least recently used*): descarta o item usado há mais tempo. Imagens em memória costumam usar esse critério.

Um pouco de história

Phil Karlton brincava que só há duas coisas difíceis na computação: invalidar cache e dar nomes às coisas.

Offline-first: o banco local como fonte da verdade



Offline-first

A interface lê e grava **sempre** no banco local. Um sincronizador troca dados com o servidor quando há rede. Sem rede, o app continua funcionando.

Fila de envio (*outbox*)

Cada alteração local vira um registro pendente. O sincronizador envia em ordem, com novas tentativas e espera crescente, mesmo com o app fechado (WorkManager, `workmanager`).

Atenção

Reenvios podem duplicar operações: mande uma chave de idempotência para o servidor reconhecer repetições (Módulo 17).

Sincronização e conflitos

O conflito

Ana e Bruno, sem internet, reservam a bancada B3 para as 14 h. Quando a rede volta, as duas reservas chegam ao servidor. Quem fica com a bancada?

Estratégia	Quando usar
Última gravação vence	dados pessoais pouco disputados; depende de relógios confiáveis
Regra no servidor	recursos disputados: a segunda reserva é recusada
Mesclagem por campo	edições em campos diferentes do mesmo registro
Perguntar ao usuário	textos editados dos dois lados
CRDT	dados colaborativos que se mesclam sozinhos

Concorrência otimista

Cada registro tem uma versão. Quem grava informa a versão que leu; se outra pessoa gravou antes, o servidor recusa e o app resolve o conflito.

Na interface

Mostre o que está pendente de envio e avise com clareza quando uma operação for recusada. Ações que não podem esperar (pagar, reservar) devem exigir rede.

Dados em repouso: criptografia e segredos

Criptografia em repouso

Os dados gravados ficam cifrados e só são decifrados com uma chave. Quem copia o armazenamento sem a chave vê apenas bytes sem sentido.

Camada	O que protege
Sistema	todo o armazenamento: criptografia por arquivo (Android) e *Data Protection* (iOS)
Cofre de chaves	chaves e segredos em hardware: Keystore (Android), Keychain e Secure Enclave (iOS)
App	banco cifrado (SQLCipher) e campos sensíveis cifrados

Por que o hardware importa

Chaves criadas no Keystore ou no Secure Enclave não saem do chip: o app pede "cifre isto" e recebe o resultado, sem nunca ver a chave.

O que nunca fazer

Guardar senhas; guardar *tokens* em preferências comuns; deixar chaves de API fixas no código, que pode ser extraído do pacote publicado.

Dica

Pense no risco: aparelho perdido, aparelho com *root* e cópias de segurança na nuvem.

Opções de armazenamento

Onde guardar cada coisa

Mecanismo	Indicado para	Exemplo no LabMonitor
DataStore Preferences	pares chave-valor pequenos	tema, unidade (°C/°F), limite de alerta
Proto DataStore	objeto de configuração tipado	perfil do usuário em cache
Room (SQLite)	dados estruturados, consultas, relações	equipamentos, bancadas, leituras
Arquivos internos	arquivos privados do app	CSV exportado, logs
Cache (`cacheDir`)	dados descartáveis	miniaturas baixadas
MediaStore / seletor	arquivos compartilhados com o usuário	fotos de equipamentos na galeria

Armazenamento interno

Pasta privada do app (`filesDir`). Outros apps não acessam, e ela é apagada na desinstalação.

SharedPreferences

A API antiga é síncrona e pode travar a UI. Em código novo, use DataStore.

DataStore

DataStore Preferences

AjustesRepository.kt

```
private val Context.ajustes by preferencesDataStore(
  name = "ajustes")

class AjustesRepository @Inject constructor(
  @ApplicationContext private val ctx: Context
) {
  private object Chaves {
    val LIMITE = doublePreferencesKey("limite_alerta")
    val FAHRENHEIT = booleanPreferencesKey("usar_f")
  }

  val limite: Flow<Double> = ctx.ajustes.data
    .catch { emit(emptyPreferences()) }  // erro de I/O
    .map { it[Chaves.LIMITE] ?: 30.0 }

  suspend fun definirLimite(valor: Double) {
    ctx.ajustes.edit { it[Chaves.LIMITE] = valor }
  }
}
```

Características

- Leitura como `Flow`: a UI atualiza sozinha.
- Escrita transacional com `edit`.
- Main-safe: I/O fora da thread principal.

Dependência

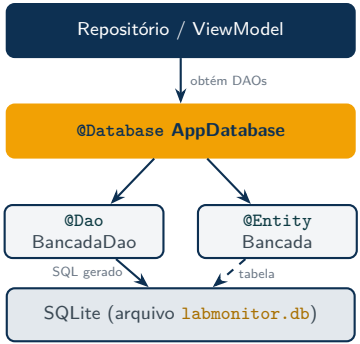
`androidx.datastore:datastore-preferences`

Atenção

Uma instância por arquivo; listas grandes vão para o Room.

Room

Arquitetura do Room



Room é uma camada sobre o SQLite que:

- verifica o SQL **em tempo de compilação**;
- converte linhas em objetos Kotlin automaticamente;
- devolve `Flow` que reemite quando a tabela muda;
- suporta coroutines e migrações;
- funciona também em Kotlin Multiplatform.

Dependências

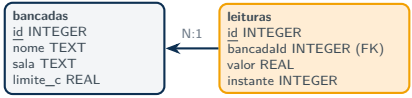
`room-runtime`, `room-ktx` e `ksp(room-compiler)`.

Entidades

Entidades.kt

```kotlin
@Entity(tableName = "bancadas")
data class BancadaEntity(
  @PrimaryKey val id: Int,
  val nome: String,
  val sala: String,
  @ColumnInfo(name = "limite_c") val limite: Double = 30.0
)

@Entity(
  tableName = "leituras",
  foreignKeys = [ForeignKey(
    entity = BancadaEntity::class,
    parentColumns = ["id"], childColumns = ["bancadaId"],
    onDelete = ForeignKey.CASCADE)],
  indices = [Index("bancadaId")]
)
data class LeituraEntity(
  @PrimaryKey(autoGenerate = true) val id: Long = 0,
  val bancadaId: Int,
  val valor: Double,
  val instante: Instant
)
```



Anotações

`@Entity` vira tabela, `@PrimaryKey` define a chave e `@ColumnInfo` renomeia colunas. `ForeignKey` garante a integridade e o `CASCADE` apaga as leituras junto com a bancada.

Dica

Crie `Index` nas colunas usadas em `WHERE` e em chaves estrangeiras.

DAO: *Data Access Object*

BancadaDao.kt

```
@Dao
interface BancadaDao {
  @Query("SELECT * FROM bancadas ORDER BY nome")
  fun observarTodas(): Flow<List<BancadaEntity>>

  @Query("SELECT * FROM bancadas WHERE id = :id")
  suspend fun buscar(id: Int): BancadaEntity?

  @Upsert                              // insere ou atualiza
  suspend fun salvar(vararg b: BancadaEntity)

  @Delete suspend fun remover(b: BancadaEntity)
  @Query("""SELECT AVG(valor) FROM leituras
    WHERE bancadaId = :id AND instante >= :desde""")
  fun media(id: Int, desde: Instant): Flow<Double?>
  @Transaction
  suspend fun substituirTodas(lista: List<BancadaEntity>) {
    apagarTodas(); salvar(*lista.toTypedArray())
  }
  @Query("DELETE FROM bancadas") suspend fun apagarTodas()
}
```

Retorno	Comportamento
`Flow<T>`	observa: reemite a cada mudança
`suspend fun`	executa uma vez, fora da UI

Parâmetros

`:id` na consulta liga-se ao parâmetro `id` da função. O Room valida nomes de tabelas e colunas na compilação.

`@Transaction`

Agrupa operações: ou todas acontecem, ou nenhuma. Evita que a UI veja a tabela vazia no meio da troca.

Banco de dados e conversores de tipo

AppDatabase.kt

```
@Database(
  entities = [BancadaEntity::class,
              LeituraEntity::class],
  version = 1,
  exportSchema = true
)
@TypeConverters(Conversores::class)
abstract class AppDatabase : RoomDatabase() {
  abstract fun bancadaDao(): BancadaDao
  abstract fun leituraDao(): LeituraDao
}

class Conversores {
  @TypeConverter
  fun deInstant(i: Instant?): Long? = i?.toEpochMilli()
  @TypeConverter
  fun paraInstant(ms: Long?): Instant? =
    ms?.let(Instant::ofEpochMilli)
}
```

Conversores

O SQLite só conhece inteiro, real, texto e blob. `@TypeConverter` ensina o Room a guardar `Instant`, `enum` e outros tipos.

Criação via Hilt (Módulo 08)

`Room.databaseBuilder(...)` em um `@Provides @Singleton`: uma instância única para o app inteiro.

Dica

`createFromAsset("inicial.db")` cria o banco já preenchido (ex.: inventário do laboratório).

Relações entre tabelas

Um-para-muitos

```
data class BancadaComEquipamentos(
  @Embedded val bancada: BancadaEntity,
  @Relation(
    parentColumn = "id",
    entityColumn = "bancadaId"
  )
  val equipamentos: List<EquipamentoEntity>
)

@Dao
interface BancadaDao {
  @Transaction
  @Query("SELECT * FROM bancadas WHERE id = :id")
  fun comEquipamentos(id: Int):
    Flow<BancadaComEquipamentos?>
}
```

`@Embedded` + `@Relation`

O Room faz duas consultas e monta o objeto composto: a bancada e a lista dos seus equipamentos.

Muitos-para-muitos

Crie uma tabela de junção (ex.: `ReservaUsuario`) e use `@Relation(associateBy = Junction(...))`.

Atenção

Use `@Transaction` em consultas com `@Relation` para que as duas consultas leiam um estado consistente.

Migrações: evoluindo o esquema

De v1 para v3

```
@Database(entities = [...], version = 3,
  autoMigrations = [AutoMigration(from = 1, to = 2)])
abstract class AppDatabase : RoomDatabase()

// mudanças complexas: migração manual
val MIGRACAO_2_3 = object : Migration(2, 3) {
  override fun migrate(db: SupportSQLiteDatabase) {
    db.execSQL("ALTER TABLE bancadas " +
      "ADD COLUMN responsavel TEXT")
    db.execSQL("CREATE INDEX idx_leitura_inst " +
      "ON leituras(instante)")
  }
}

Room.databaseBuilder(ctx, AppDatabase::class.java, "lab.db")
  .addMigrations(MIGRACAO_2_3)
  .build()
```

Regra

Mudou uma entidade? **Incremente a versão** e forneça a migração, senão o app trava ao abrir o banco antigo.

`exportSchema`

Gera um JSON do esquema de cada versão. Versione esses arquivos: são a base das *auto migrations* e dos testes de migração.

Erro comum

`fallbackToDestructiveMigration()` apaga os dados do usuário. Aceitável só em protótipos ou caches.

Integração com a arquitetura

Fluxo reativo de ponta a ponta



A mágica

Depois que o DAO grava, o Room **notifica** todos os `Flow` que leem aquela tabela. A tela se atualiza sem nenhum código extra de "recarregar".

Inspecionando

App Inspection → **Database Inspector** mostra as tabelas do app em execução, permite rodar SQL e atualiza ao vivo.

Guardando leituras de sensores

LeituraRepository.kt

```
class LeituraRepository @Inject constructor(
  private val dao: LeituraDao
) {
  fun ultimas(bancada: Int, n: Int = 120) =
    dao.ultimas(bancada, n)          // Flow

  suspend fun registrar(l: Leitura) =
    dao.inserir(l.paraEntidade())

  // mantém o banco pequeno: apaga o que é antigo
  suspend fun limpar(dias: Long = 30) =
    dao.apagarAntesDe(
      Instant.now().minus(dias, ChronoUnit.DAYS))
}

@Query("""SELECT * FROM leituras WHERE bancadaId = :b
  ORDER BY instante DESC LIMIT :n""")
fun ultimas(b: Int, n: Int): Flow<List<LeituraEntity>>
```

Volume de dados

Um sensor a 1 Hz gera 86 400 linhas por dia. Planeje:

- retenção (apagar o que é antigo);
- agregação (médias por minuto);
- consultas com `LIMIT` e índices.

Dica

Para inserir muitas leituras, agrupe em lote (`@Insert` com `List`) em vez de uma transação por leitura.

Boas práticas e segurança

Boas práticas

- Nunca acesse o banco na thread principal (o Room bloqueia por padrão).
- Uma instância de `AppDatabase` por processo.
- Teste as migrações com `MigrationTestHelper`.
- Banco em memória (`inMemoryDatabaseBuilder`) nos testes (Módulo 14).

Dados sensíveis

- Não guarde senhas. Guarde *tokens* e prefira o **Android Keystore** para chaves criptográficas.
- O backup automático do Android copia arquivos do app para a nuvem do usuário; exclua o que não deve sair do aparelho com `dataExtractionRules`.
- Trate dados pessoais conforme a **LGPD** (Módulo 15).

Fechamento

Resumo

Resumo

- Preferências pequenas no DataStore; dados estruturados no Room; arquivos em `filesDir`.
- Room = `@Entity` (tabelas) + `@Dao` (consultas) + `@Database` (o banco).
- DAOs devolvem `Flow` para observar e usam `suspend` para operações pontuais.
- `@TypeConverter` para tipos especiais; `@Relation` para relações.
- Toda mudança de esquema exige nova versão e migração.
- Com repositório + `Flow`, gravar no banco atualiza a tela automaticamente.

Glossário (1/2)

Termo	Significado
Persistência	guardar dados que sobrevivem ao fim do processo e ao desligar
***Sandbox* de arquivos**	pastas privadas do app, apagadas na desinstalação
DataStore	armazenamento chave-valor assíncrono, baseado em `Flow`
Room	camada sobre o SQLite com SQL verificado na compilação
`@Entity`	classe que representa uma tabela
DAO	interface com as operações de acesso aos dados
Migração	leva o banco de uma versão do esquema à seguinte sem perder dados
Descompasso objeto-relacional	objetos têm referências e listas; tabelas têm linhas e chaves
Problema N+1	uma consulta para a lista e mais uma para cada item

Glossário (2/2)

Termo	Significado
Cache	cópia guardada perto de quem usa, para não buscar de novo
Invalidação	decidir quando o cache deixou de valer (TTL, evento, versão)
LRU	política que descarta o item usado há mais tempo
Offline-first	a interface lê e grava no banco local; a sincronização vem depois
Fila de envio	alterações locais pendentes de envio ao servidor (*outbox*)
Concorrência otimista	a gravação é recusada se a versão lida já mudou
Android Keystore	cofre de chaves do Android, protegido por hardware

Questões de revisão

1. Por que um app precisa persistir dados mesmo quando tem internet?
2. Onde guardar a preferência de tema, o *token* de acesso, o histórico de leituras e as miniaturas baixadas?
3. Que vantagens o Room traz sobre escrever SQL à mão com `SQLiteOpenHelper`?
4. Um usuário atualiza da versão 1 direto para a 4 do app. O que o Room precisa ter para não perder os dados?
5. Explique o problema N+1 e como evitá-lo no Room.
6. Descreva o caminho de uma reserva feita sem internet em uma arquitetura *offline-first*.
7. Dois alunos reservam a mesma bancada sem internet. Que estratégia de conflito usar e por quê?
8. Por que não guardar o *token* em preferências comuns? Onde deve ficar a chave?

Exercícios

Exercício 1 — DataStore

Crie a tela de Ajustes com limite de alerta (`Slider`), unidade de temperatura (`Switch`) e intervalo de amostragem. Todos os valores devem persistir.

Exercício 2 — Consultas

Escreva consultas DAO para: as 5 bancadas com maior média de temperatura nas últimas 24 h; a contagem de leituras acima do limite por bancada; leituras entre duas datas.

Exercício 3 — Migração

Adicione a coluna `responsavel` à tabela de bancadas e escreva a migração 1→2. Confirme no Database Inspector que os dados antigos foram preservados.

LabMonitor — Etapa 10

Substitua os repositórios *fake* por Room: bancadas, equipamentos e leituras do simulador. Guarde o limite de alerta no DataStore e implemente a limpeza de leituras com mais de 30 dias.

Para aprofundar

- Visão geral de armazenamento: `https://developer.android.com/training/data-storage`
- DataStore: `https://developer.android.com/topic/libraries/architecture/datastore`
- Room: `https://developer.android.com/training/data-storage/room`
- Relações no Room:
  `https://developer.android.com/training/data-storage/room/relationships`
- Migrações:
  `https://developer.android.com/training/data-storage/room/migrating-db-versions`
- Backup automático: `https://developer.android.com/identity/data/autobackup`

Respostas das questões de revisão

1. O sistema encerra apps em segundo plano, a rede falha e o usuário quer ver os dados na hora: o banco local serve de cache e de fonte da verdade.
2. Tema: DataStore Preferences. *Token*: cifrado com chave do Android Keystore. Leituras: Room. Miniaturas: `cacheDir`.
3. Consultas verificadas na compilação, código de conversão gerado, suporte a `Flow`, relações e migrações.
4. As migrações 1→2, 2→3 e 3→4 registradas no `databaseBuilder` (ou *auto migrations*); o Room as aplica em sequência. Teste cada uma com `MigrationTestHelper`.
5. Buscar a lista e depois fazer uma consulta por item gera N+1 idas ao banco. No Room, use `@Relation` com `@Transaction` ou um `JOIN` em uma só consulta.
6. A reserva é gravada no Room como pendente, junto com um registro na fila de envio, na mesma transação. A tela já a mostra; o WorkManager a envia quando houver rede e atualiza o estado com a resposta.
7. Regra no servidor: a bancada é um recurso disputado. A primeira reserva confirmada vence, a segunda é recusada e o app avisa o usuário.
8. O app não cifra esse arquivo, que pode ir para cópias de segurança ou ser lido em aparelhos com *root*. A chave deve ficar no Android Keystore, de onde não sai.

Fim do Módulo 10

Próximo: Módulo 11 — Rede e consumo de APIs: HTTP, REST, Retrofit, serialização, erros e estratégia offline-first.